

Privacy Node Build

Running Bitcoin, Monero, and Tor on a \$264 Machine

January 2026

Contents

1	The Goal	2
2	Hardware	2
3	Service Architecture	2
4	Phase 1: Base System Setup	3
5	Phase 2: Storage Infrastructure	4
6	Phase 3: Tor Snowflake Proxy	5
7	Phase 4: Bitcoin Core Node	6
8	Phase 5: Monero Pruned Node	7
9	Current Status	9

1 The Goal

I wanted to run three distinct services on a single always-on machine: a full Bitcoin archival node, a pruned Monero node, and a Tor Snowflake proxy. The idea was to actually contribute something useful to the networks I care about — not just install software and call it a day, but run real infrastructure that serves real users.

A full Bitcoin archival node stores and serves every transaction in the blockchain’s history, making it a first-class peer that other nodes can sync from rather than a passive participant. A pruned Monero node validates all transactions without retaining the full transaction history, giving the Monero network an additional validation peer at a fraction of the disk cost. The Tor Snowflake proxy uses WebRTC to serve as an entry bridge for users in censored regions who cannot otherwise reach the Tor network.

2 Hardware

Component	Specification
Machine	Dell OptiPlex 7020 Tower
CPU	Intel Core i7-4790 (Quad-core, 3.6 GHz)
RAM	32 GB DDR3
Storage A (OS)	512 GB SSD (<code>/dev/sda</code>)
Storage B (Data)	3 TB HDD (<code>/dev/sdb</code>)
Network	Intel AC8260 WiFi + Gigabit Ethernet
Power	~40–50 W idle ($\approx$ \$5/month)
Hardware Cost	\$264

Table 1: Build specifications

The ISP is a symmetrical gigabit fiber connection with no data cap. Confirming the absence of a data cap was a hard prerequisite before committing to this build, because both the Bitcoin blockchain sync and an uncapped Snowflake proxy push significant traffic continuously.

3 Service Architecture

Three independent services run simultaneously on the node, each managed by `systemd` and isolated to its own dedicated system user where applicable.

Bitcoin Core (`bitcoind`)

Bitcoin Core runs as a daemon under a dedicated user. It binds to port `8333` for peer-to-peer traffic and stores the full blockchain on the dedicated 3 TB HDD mounted at `/mnt/bitcoin`. The node operates in archival mode (`prune=0`), retaining the complete UTXO set and all

historical blocks, which allows it to serve chain data to syncing peers. The RPC interface is enabled locally for administrative queries but is not exposed to the network.

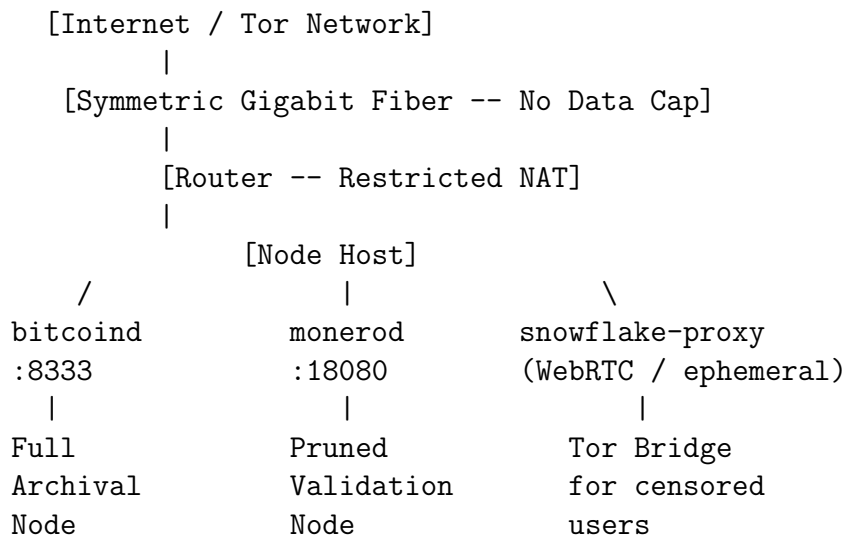
Monero (monerod)

The Monero daemon runs as a dedicated system user with no login shell, following the principle of least privilege. It validates the full Monero transaction graph but operates in pruned mode (`--prune-blockchain`), discarding redundant historical chain data after validation while preserving all consensus-relevant information. This reduces the storage footprint to approximately 100 GB rather than the ~220 GB required for a full unpruned node. `monerod` binds to port 18080 for P2P traffic.

Tor Snowflake Proxy (snowflake-proxy)

The Snowflake proxy uses WebRTC — the same protocol that powers browser video calls — to ferry traffic for censored users into the Tor network without requiring any inbound port forwarding. The service runs under a dedicated system user and is registered with `systemd` to start at every boot.

Network Topology



All three services start automatically on boot via `systemd`. The machine runs headless, with no SSH daemon installed. All administration is performed physically.

4 Phase 1: Base System Setup

Ubuntu Server 24.04.3 LTS was installed on the 512 GB SSD using manual full-disk partitioning. LVM was deliberately avoided — the volume layout is simple and static, and LVM adds unnecessary abstraction with no benefit for this use case.

Security hardening was addressed before any service was installed:

- A single administrative user was created with a strong password.
- The hostname was set to a generic, unremarkable string to avoid advertising the machine's purpose on the local network.
- **SSH was not installed.** The machine is managed physically only (keyboard and monitor). This eliminates an entire class of remote attack surface at the cost of minor inconvenience.
- The system was fully patched prior to any additional software installation:

```
sudo apt update && sudo apt upgrade -y
```

5 Phase 2: Storage Infrastructure

The secondary 3 TB HDD (`/dev/sdb`) was provisioned as dedicated storage for the Bitcoin blockchain. The goal was a clean, persistent mount that would survive reboots without manual intervention.

Partitioning

The drive was wiped and a single GPT partition spanning the full disk was created, formatted as `ext4`:

```
sudo parted /dev/sdb mklabel gpt
sudo parted /dev/sdb mkpart primary ext4 0% 100%
sudo mkfs.ext4 /dev/sdb1
```

Mount Configuration

The mount point was created and ownership was assigned to the administrative user:

```
sudo mkdir -p /mnt/bitcoin
sudo mount /dev/sdb1 /mnt/bitcoin
sudo chown <admin-user>:<admin-user> /mnt/bitcoin
```

To persist the mount across reboots, the drive's UUID was retrieved and written to `/etc/fstab`:

```
# Retrieve the UUID
sudo blkid /dev/sdb1

# Add to /etc/fstab (replace UUID with actual value):
UUID=<your-uuid> /mnt/bitcoin ext4 defaults 0 2
```

After a reboot, the mount was verified with `df -h`, confirming approximately 2.7 TB available on `/mnt/bitcoin`.

6 Phase 3: Tor Snowflake Proxy

Why Not a Middle Relay?

The original plan was to run a Tor middle relay. Middle relays require a stable public-facing port (typically 9001) and benefit from a static IP. A quick diagnostic revealed a restricted NAT on the residential fiber connection, meaning inbound port forwarding was not straightforwardly available. Rather than fight the router configuration, the decision was made to pivot to a Snowflake proxy, which is architecturally designed for exactly this network environment.

Installation

The `snowflake-proxy` package is available directly from the standard Ubuntu repositories:

```
sudo apt install snowflake-proxy -y
```

Configuration

An important configuration decision came up during setup. The initial service configuration included a `-capacity` flag to cap the number of simultaneous clients, motivated by uncertainty about whether the ISP imposed a data cap. Once the billing statement confirmed no data cap on the fiber connection, the flag was removed entirely:

```
# Enable and start the service (no -capacity flag = uncapped)
sudo systemctl enable snowflake-proxy
sudo systemctl start snowflake-proxy
```

Verification

The service came up cleanly. Log output confirmed a successful WebRTC handshake:

```
sudo journalctl -u snowflake-proxy -f

# Confirming operational status:
# SDP offer successfully received
# NAT type: Restricted
```

A restricted NAT type is normal and expected for a residential connection. The Snowflake protocol is specifically designed to operate under these conditions.

7 Phase 4: Bitcoin Core Node

Installation

Bitcoin Core binaries were downloaded directly from `bitcoin.org` and the SHA256 checksum was verified against the published hash before proceeding:

```
# Download (replace version with current release)
wget https://bitcoin.org/bin/bitcoin-core-<version>/\
bitcoin-<version>-x86_64-linux-gnu.tar.gz

# Verify checksum before extracting
sha256sum bitcoin-<version>-x86_64-linux-gnu.tar.gz

# Extract and install to system PATH
tar -xvf bitcoin-<version>-x86_64-linux-gnu.tar.gz
sudo mv bitcoin-<version>/bin/* /usr/local/bin/
```

Configuration

The Bitcoin data directory was pointed at the 3 TB mount, and the node was configured to run as a full archival peer:

```
mkdir -p ~/.bitcoin
nano ~/.bitcoin/bitcoin.conf
```

```
# bitcoin.conf
datadir=/mnt/bitcoin
prune=0
server=1
listen=1
daemon=1
```

`prune=0` is what makes this an archival node. It retains every block since genesis, allowing the node to serve historical chain data to peers that are syncing for the first time.

Systemd Service

```
sudo nano /etc/systemd/system/bitcoind.service
```

```
[Unit]
Description=Bitcoin Core Daemon
After=network-online.target
Wants=network-online.target
```

```
[Service]
User=<bitcoin-user>
ExecStart=/usr/local/bin/bitcoind \
    -conf=/home/<bitcoin-user>/.bitcoin/bitcoin.conf
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable bitcoind
sudo systemctl start bitcoind
```

Initial Blockchain Sync

The initial block download (IBD) is the longest part of this entire build by a significant margin. The Bitcoin blockchain currently sits at roughly 700+ GB. On an HDD rather than an SSD, the sync is dominated by random I/O during UTXO set construction rather than by network speed — gigabit fiber is not the bottleneck. The sync proceeded in two distinct phases: the pre-2017 blocks download relatively quickly because they are small, then throughput drops sharply once it reaches the larger modern blocks. On this hardware, the full IBD took approximately 10–14 days from a cold start to a fully synced chain tip. Progress can be monitored in real time:

```
bitcoin-cli getblockchaininfo
```

The `verificationprogress` field will read 1.0 when the node is fully synced.

8 Phase 5: Monero Pruned Node

Installation

A dedicated system user with no login shell was created to run `monerod` under the principle of least privilege:

```
sudo useradd --system --no-create-home --shell /sbin/nologin
    monero
```

The Monero CLI binaries were downloaded from getmonero.org and verified against the published SHA256 hash before installation:

```
# Download (replace version with current release)
wget https://downloads.getmonero.org/cli/linux64
```

```
tar -xvf monero-linux-x64-*.tar.bz2
sudo mv monero-x86_64-linux-gnu-*/ * /usr/local/bin/
sudo chown monero:monero /usr/local/bin/monero*
```

Configuration and Systemd Service

monerod was configured to run in pruned mode and registered as a systemd service:

```
sudo nano /etc/systemd/system/monerod.service
```

```
[Unit]
Description=Monero Daemon
After=network-online.target
Wants=network-online.target

[Service]
User=monero
ExecStart=/usr/local/bin/monerod \
    --prune-blockchain \
    --data-dir /var/lib/monero \
    --non-interactive \
    --log-level 0
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

```
sudo mkdir -p /var/lib/monero
sudo chown monero:monero /var/lib/monero

sudo systemctl daemon-reload
sudo systemctl enable monerod
sudo systemctl start monerod
```

The `--prune-blockchain` flag reduces disk usage to approximately 95–100 GB while preserving full transaction validation capability. Pruning does not reduce privacy or consensus correctness; it only discards redundant historical chain data that has already been validated.

Initial Blockchain Sync

Monero's initial sync is substantially faster than Bitcoin's, both because the chain is smaller and because pruned mode reduces write pressure during the sync itself. On this hardware, the pruned node reached chain tip in approximately 24–36 hours. Unlike Bitcoin's IBD,

where the bottleneck is random I/O on an HDD, Monero’s sync is more evenly balanced between CPU (ring signature verification) and disk. Progress can be monitored directly in the `monerod` log output or via:

```
sudo journalctl -u monerod -f
```

The node is fully synced when the log reports a block height matching the current Monero network tip.

9 Current Status

As of January 2026, the base system is online and fully patched. The Tor Snowflake proxy is running uncapped with a confirmed operational WebRTC handshake. Bitcoin Core and Monero are configured, and the initial blockchain syncs have completed.

Service	Status	Notes
Ubuntu Server 24.04	Online	Fully patched, no SSH
Snowflake Proxy	Running	Uncapped, SDP handshake confirmed
Bitcoin Core	Synced	Full archival node, <code>prune=0</code>
Monero	Synced	Pruned node, ~100 GB footprint

Table 2: Service status

Total hardware cost: \$264. Electricity: approximately \$5 per month. For what this machine contributes to three separate open networks, the economics are hard to argue with.